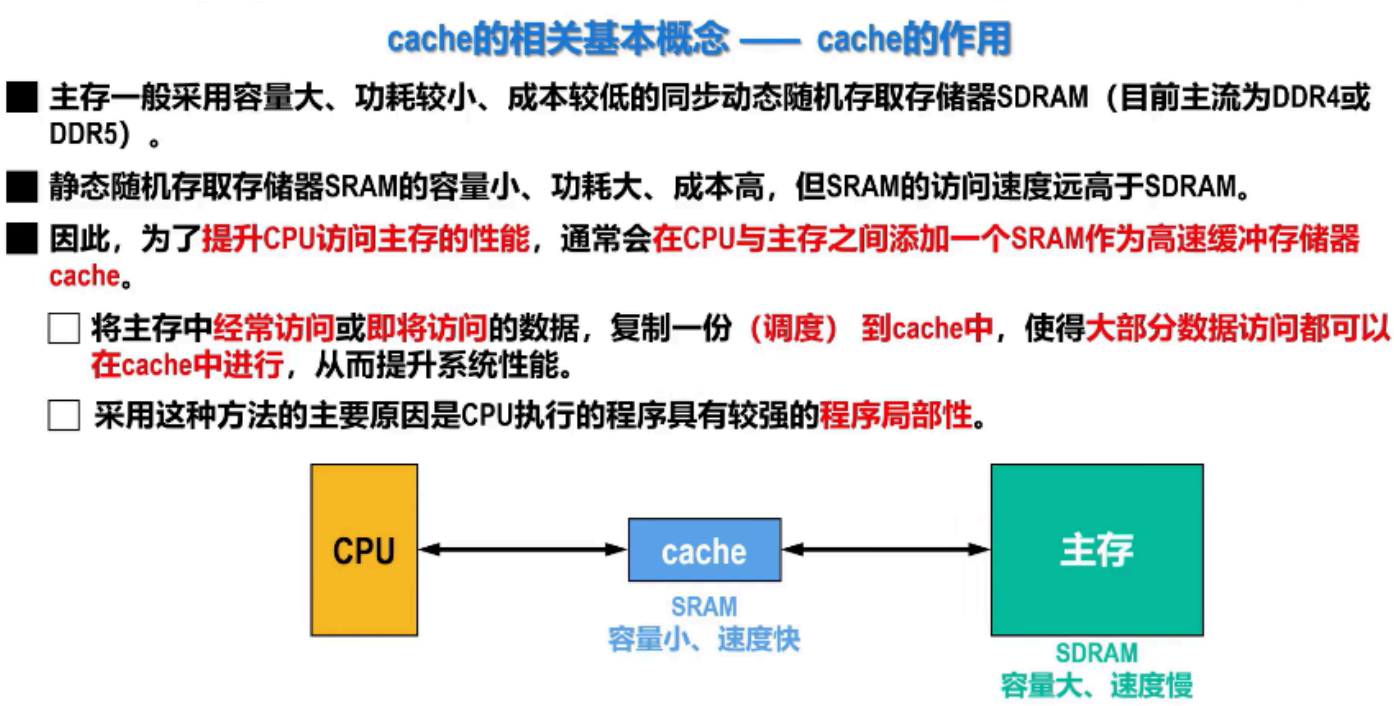
4.21 Cache——基本概念

考纲定位：计算机组成原理 · 第 4 章 存储器（4.21） | 重要度 ★★★★★ | 常见考法：选择题/大题（局部性、命中率、平均访问时间、Cache 容量、块大小）

一句话核心：Cache 是 CPU 与主存之间的 SRAM 缓冲，靠程序局部性把常用数据复制到高速小容量存储器

一、作用



提炼

- 主存用 DRAM，容量大、成本低，但速度跟不上 CPU
- Cache 用 SRAM，容量小、成本高，但速度快
- Cache 存放主存中“经常访问或即将访问”的数据副本
- CPU 先访问 Cache，命中则直接取数，未命中再访问主存

易错

- Cache 不是“主存的替代品”，而是主存的副本
- Cache 对程序员透明，由硬件自动管理
- Cache 的内容是主存内容的子集，不是全部

二、程序局部性

cache的相关基本概念 —— 程序局部性

■ 程序局部性是指，在一段时间内，整个程序的执行仅限于一个较小的局部范围内。具体来说，程序的局部性又表现为以下两种：

- □ 时间局部性：若程序在某个时刻访问了一个存储位置，该位置在未来可能会被多次访问。
  - ◇ 例如程序中的循环结构和调用过程就很好地体现了时间局部性。
- □ 空间局部性：若程序访问了某个存储位置，则其附近的存储位置也可能被访问。
  - ◇ 例如程序中的数组、结构体成员、顺序执行的代码块，通常在主存中是按顺序存放的，对它们的访问，就具有较强的空间局部性。

【举例】请尝试分析以下C语言程序代码的程序局部性。

```
for (i=0; i<1000; i++)  sum+=a[i];
```

分析

for循环中的指令序列（读取、加法）在主存中顺序存放，具有空间局部性；

for循环中的指令序列被执行1000次，具有时间局部性；

变量i、sum是单个变量，不存在空间局部性；由于其在每次执行for循环时都会被用到，因此具有时间局部性。

数组a包含至少1000个数据元素，这些数据在主存中顺序存放，具有空间局部性；数组a中的每个数据元素都只被访问一次，因此不存在时间局部性。

两类局部性

- 时间局部性：刚被访问的数据/指令，近期可能再次被访问
- 空间局部性：刚被访问的地址附近，近期可能被访问

例题

【2017年 题14】某C语言程序段如下：

```
for (i=0; i<=9; i++) {
    temp = 1;
    for (j=0; j<=i; j++)  temp *=  a[j];
    sum += temp;
}
```

下列关于数组a的访问局部性的描述中，正确的是（A）。

A. 时间局部性和空间局部性皆有

B. 无时间局部性，有空间局部性

C. 有时间局部性，无空间局部性

D. 时间局部性和空间局部性皆无

解析

数组a中的各元素在主存中顺序存放，具有空间局部性；

数组a的读取序列：a[0]、a[0]、a[1]、a[0]、a[1]、a[2]、…，

数组a中的同一元素被多次访问，具有时间局部性。

提炼

- 循环、数组、顺序指令流是局部性的典型来源
- Cache 按“块”调入，一次取一块，利用空间局部性
- 时间局部性靠保留最近访问的块

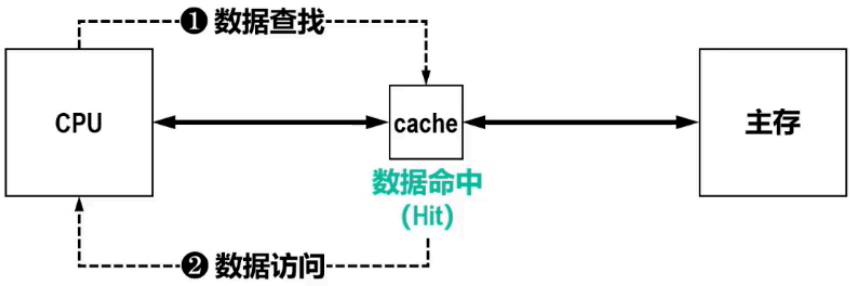
易错

- 局部性是程序行为，不是 Cache 的属性
- 时间局部性不等于“数据永远不变”
- 空间局部性要求块内数据连续；跳跃访问会降低命中率

三、访问时间

3.1 命中缓存时

■ 在CPU和主存之间添加了cache后，CPU不再直接访问慢速的主存，而是通过字节地址访问快速的cache。



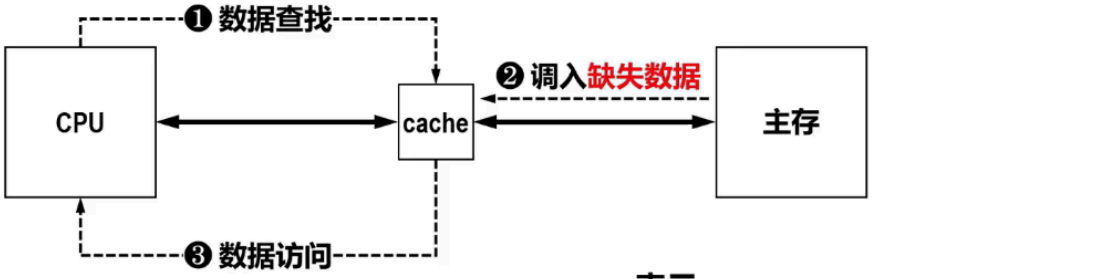
上述情况下，数据访问时间称为命中访问时间，记为t_c { cache内的查找时间；cache访问时间

提炼

- 命中时只需访问 Cache，时间记为 t_c
- 命中率 h 越高，平均访问时间越接近 t_c

3.2 未命中缓存时（数据缺失）

■ 为了便于快速查找，主存和cache都被划分成若干个固定大小的数据块（Block），每个数据块又包含若干个字。



表示

上述情况下，数据访问时间称为缺失补偿（Miss Penalty） { cache内的查找时间；主存访问时间（较为漫长），记为t_m；cache访问时间

提炼

- 未命中时要从主存调入数据，时间记为 t_m
- 若采用“先访问 Cache 再访问主存”，平均访问时间公式为

$$t_a = ht_c + (1-h)t_m$$

- 若采用“同时访问 Cache 和主存”，未命中时只需补上主存时间，公式会不同；408 默认前者

易错

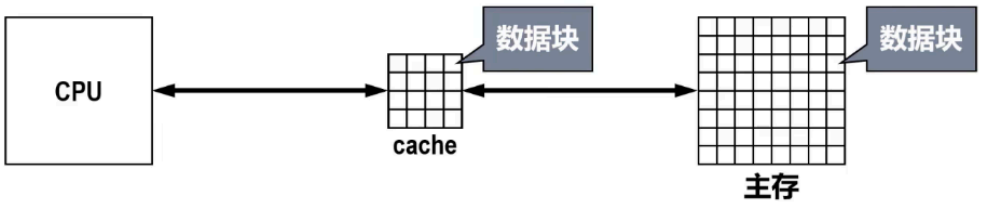
- t_m 是未命中时的总时间，可能包含主存访问 + 调入 Cache
- 命中率与缺失率之和为 1
- 公式中的 h 是命中率，不是缺失率

四、数据分块

■ 为了便于快速查找，主存和cache都被划分成若干个固定大小的数据块（Block），每个数据块又包含若干个字。

□ 若进行数据访问时出现数据缺失的情况，则需要将缺失数据所在的数据块从慢速主存载入cache中。

□ 因此，缺失数据相邻的数据也会随着数据块一起载入cache。

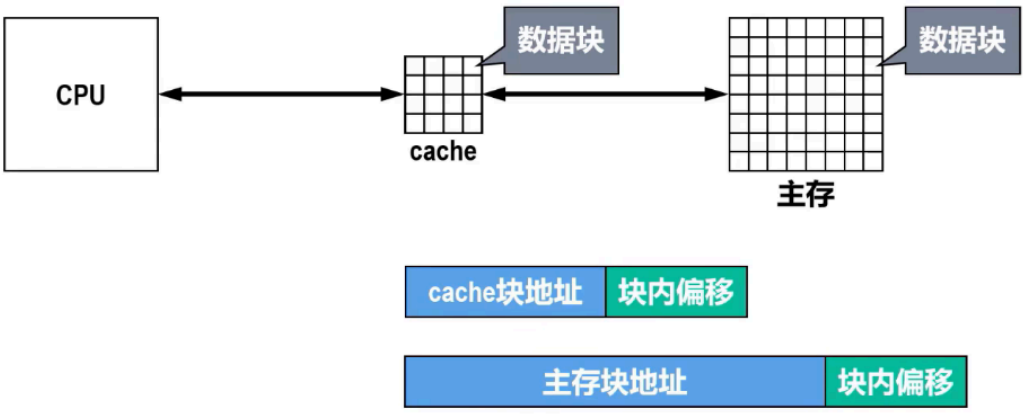


- 上述预读策略可以充分利用程序的空间局部性，提高顺序访问的命中率。
- 然而，数据块的大小对cache有较大影响：
  - 数据块过小：无法利用预读策略优化程序的空间局部性。
  - 数据块过大：将使得替换算法无法充分利用程序的时间局部性。

■ 进行数据分块后，需要给主存内的数据块和cache内的数据块分配地址。

□ 数据块的地址由块地址和块内偏移地址（简称块内偏移Offset）两部分构成。

□ 由于cache容量比主存容量小很多，因此cache内的块地址长度小于主存内的块地址长度。



提炼

- Cache 与主存按“块”交换数据，块大小相同
- 块大小 = 一次调入 Cache 的数据量
- 块太小：空间局部性利用不足，命中率低
- 块太大：调入开销大，冲突概率增加，可能降低命中率

- 块大小通常为 4~64 字节

易错

- Cache 行（line）与主存块（block）通常一一对应
- 块大小不是越大越好；要折中
- 块内地址偏移位数 $= \log_2(\text{块大小})$

五、评价指标

某程序运行期间命中cache的次数记为 n_c，从主存中访问信息的次数记为n_m，命中率（Hit Ratio）记为h：

$$h = \frac{n_c}{n_c + n_m}$$

h越接近于1性能越好

$1-h$ 称为缺失率（Miss Ratio）。命中情况下的访问时间记为t_c，数据缺失情况下的访问时间记为t_m，cache/主存系统的平均访问时间记为t_a：

$$t_a = ht_c + (1-h)t_m$$

h越接近于1，t_a越接近t_c

访问效率记为e：

$$e = \frac{t_c}{t_a} = \frac{t_c}{ht_c + (1-h)t_m} = \frac{1}{h + (1-h)\frac{t_m}{t_c}} = \frac{1}{h + (1-h)r}$$

h越接近于1，e越接近1

r一般为5~10，不能太大

公式

$$h = \frac{n_c}{n_c + n_m}$$

$$t_a = ht_c + (1-h)t_m$$

$$e = \frac{t_c}{t_a} = \frac{1}{h + (1-h)\frac{t_m}{t_c}}$$

- n_c：命中次数；n_m：未命中次数
- t_c：命中时间；t_m：未命中时间
- e：访问效率，越接近 1 越好

例：$h = 0.95$，$t_c = 1ns$，$t_m = 100ns$

$$t_a = 0.95 \times 1 + 0.05 \times 100 = 5.95ns$$

$$e = 1/5.95 \approx 16.8\%$$

易错

- 平均访问时间不是 $ht_c + (1-h)t_c$
- 命中率很高但 t_m 很大时，平均时间仍可能被拖慢
- Cache 容量、块大小、映射方式、替换算法都会影响命中率

知识关联：存储层次（4.1 存储器概述）； 性能指标（4.3 存储器的性能指标和存储系统层次结构）； 直接映射（4.23 Cache——地址映射（直接映射））； 替换算法（4.26 Cache——替换算法）